

Artificial Intelligence and Machine Learning(6CS012)

Part - III - Language Tasks

Name: Mandeep Panta

Student Id: 2408214

Lecturer/Tutor: Ms. Sunita Parajuli

Module Leader: Mr. Siman Giri

Submission Date: 2026-05-10

Table of Contents

Introduction	1
Dataset.....	2
Methodology.....	3
Model Architecture	3
Experiments and Results	6
RNN vs. LSTM Performance	6
Computational Efficiency	6
Training with Different Embeddings	7
Model Evaluation	7
Evaluation Summary:.....	8
Conclusion and Future Work.....	9

Abstract

This project addresses the critical task of classifying Short Message Service (SMS) messages as either 'spam' or 'ham' (legitimate) to enhance user experience and

security by filtering unwanted communications. We employed deep learning techniques, specifically Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, utilizing both randomly initialized embeddings and pre-trained Word2Vec embeddings for text representation. The methodology involved comprehensive text preprocessing, including cleaning, tokenization, stopwords removal, and lemmatization, followed by sequence padding. Our experiments revealed that the standalone LSTM model achieved the highest accuracy of 0.9821 and a strong recall for spam messages (0.87), demonstrating its superior performance compared to the Simple RNN (0.9758 accuracy, 0.84 spam recall). Interestingly, the Word2Vec + LSTM model performed poorly with an accuracy of 0.8664 and a recall of 0.00 for spam, suggesting that the custom-trained Word2Vec on this specific dataset was not effective. In conclusion, LSTM networks prove highly effective for this text classification task, accurately identifying both spam and ham messages. Future work could involve hyperparameter optimization, leveraging larger external pre-trained embeddings like GloVe or FastText, and exploring more complex model architectures to further improve spam detection capabilities.

Introduction

The problem statement for this project is the binary classification of SMS messages into 'spam' or 'ham' categories. This task is of significant real-world importance as it helps protect users from unsolicited, potentially malicious, or unwanted messages, thereby improving communication efficiency and security. Deep learning models, particularly Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, are highly relevant for sequence data like text due to their ability to process sequential information and capture long-range dependencies. Text classification has been a long-standing area of research, with traditional machine learning methods evolving into sophisticated deep learning approaches that can automatically learn hierarchical features from raw text.

Dataset

Source: The dataset used for this project is spamvsham.csv, provided by Tutor Ms. Sunita Parajuli.

Data Size: The initial dataset contained 5572 messages. After preprocessing and cleaning, the final dataset also retained 5572 entries, comprising two classes: 'ham' (mapped to 0) and 'spam' (mapped to 1).

Preprocessing: The following preprocessing steps were applied to the raw SMS messages:

- Column Selection and Renaming: Kept the first two columns and renamed them to 'label' and 'message'.
- Label Cleaning: Converted labels to lowercase, stripped whitespace, and mapped 'ham' to 0 and 'spam' to 1.
- Text Cleaning Function (clean_text):
 - Converted all text to lowercase.
 - Expanded contractions (e.g., "don't" to "do not").
 - Removed URLs, mentions (@user), and hashtags (#tag).
 - Removed numbers.
 - Removed special characters and punctuation.
 - Tokenization: Broke down sentences into individual words using nltk.word_tokenize.
 - Stopword Removal: Eliminated common English stopwords (e.g., "the", "is", "a").
 - Lemmatization: Reduced words to their base or root form (e.g., "running" to "run").

Methodology

Text Preprocessing

After the initial cleaning, the text data underwent further preparation for deep learning models:

- **Tokenization:** A Tokenizer from Keras was used to convert words into numerical sequences. MAX_WORDS was set to 5000, and out-of-vocabulary tokens were handled with "". The tokenizer was fitted only on the training data.
- **Padding:** To ensure uniform input length for the neural networks, sequences were padded. The MAX_LENGTH for padding was determined by the 95th percentile of training sequence lengths, which was found to be 18. Sequences shorter than MAX_LENGTH were padded with zeros ('post' padding), and longer sequences were truncated ('post' truncating).

Model Architecture

All models utilized an Embedding layer at the input and a Dense layer with a sigmoid activation for binary classification at the output. MAX_LENGTH was the input shape.

- **Simple RNN (model1):**
 - Input Layer.
 - Embedding Layer: input_dim=MAX_WORDS, output_dim=64 (embedding_dim).
 - SimpleRNN Layer: 64 units.
 - Dropout Layer: 0.3.
 - Dense Layer: 1 unit with 'sigmoid' activation.
- **LSTM (model2):**
 - Input Layer.
 - Embedding Layer: input_dim=MAX_WORDS, output_dim=64 (embedding_dim).

- LSTM Layer: 64 units.
- Dropout Layer: 0.3.
- Dense Layer: 1 unit with 'sigmoid' activation.
- **Word2Vec + LSTM (model3):**
 - Word2Vec Training: A Word2Vec model was trained on the tokenized training sentences (sentences = [text.split() for text in X_train]) with vector_size=100, window=5, min_count=1, and workers=4. An embedding_matrix was then created, initializing the embeddings for words present in the Word2Vec model.
 - Input Layer.
 - Embedding Layer: input_dim=MAX_WORDS, output_dim=100 (embedding_dim_w2v), weights=[embedding_matrix], and trainable=False (to keep Word2Vec embeddings fixed).
 - LSTM Layer: 64 units.
 - Dropout Layer: 0.3.
 - Dense Layer: 1 unit with 'sigmoid' activation.

Loss Function

For all models, **'binary_crossentropy'** was used as the loss function, which is standard for binary classification tasks.

Optimizer

The **'adam'** optimizer was used for compiling all models, known for its efficiency and good performance in a wide range of deep learning tasks.

Hyperparameters

- Epochs: EPOCHS = 20 (though early stopping often halts training sooner).
- Batch Size: batch_size = 32.

- Embedding Dimension: embedding_dim = 64 for Simple RNN and LSTM with random embeddings; embedding_dim_w2v = 100 for Word2Vec + LSTM.
- Callbacks:
 - EarlyStopping: monitor='val_loss', patience=3, restore_best_weights=True.
 - ReduceLROnPlateau: monitor='val_loss', factor=0.5, patience=3.
 - ModelCheckpoint: filepath="best_model.keras", monitor="val_accuracy", save_best_only=True.

Experiments and Results

RNN vs. LSTM Performance

Model	Accuracy	Precision (Ham)	Recall (Ham)	F1-Score (Ham)	Precision (Spam)	Recall (Spam)	F1-Score (Spam)
Simple RNN	0.9758	0.98	1.00	0.99	0.98	0.84	0.90
LSTM	0.9821	0.98	1.00	0.99	0.99	0.87	0.93

The LSTM model (model2) demonstrated superior performance compared to the Simple RNN (model1). While both models achieved high overall accuracy, LSTM slightly surpassed Simple RNN (0.9821 vs. 0.9758). More importantly, the LSTM model showed better recall for the 'spam' class (0.87 vs. 0.84), indicating its improved ability to correctly identify actual spam messages. This is a crucial advantage in spam detection, where minimizing false negatives (misclassifying spam as ham) is often a priority. The LSTM's ability to capture longer-term dependencies in text sequences likely contributed to this improvement over the simpler RNN architecture.

Computational Efficiency

The training times were measured for each model. Generally, Simple RNNs are computationally less expensive than LSTMs due to their simpler architecture. LSTMs, with their gating mechanisms, involve more computations per time step, leading to longer training times. The Word2Vec + LSTM model also involved an initial step of training the Word2Vec embeddings, which adds to the overall computational overhead. However, the exact training times for each model in this specific run were

not explicitly captured in the outputs, but the relative performance aligns with typical deep learning models.

Training with Different Embeddings

Model	Accuracy	Precision (Ham)	Recall (Ham)	F1-Score (Ham)	Precision (Spam)	Recall (Spam)	F1-Score (Spam)
Simple RNN (Random)	0.9758	0.98	1.00	0.99	0.98	0.84	0.90
Word2Vec + LSTM	0.8664	0.87	1.00	0.93	0.00	0.00	0.00

Surprisingly, the Word2Vec + LSTM model (model3) performed significantly worse than the models using randomly initialized embeddings (Simple RNN and LSTM). The Word2Vec + LSTM achieved a much lower accuracy of 0.8664 and completely failed to detect any spam messages, exhibiting a 0.00 recall for the 'spam' class. This indicates that the Word2Vec embeddings, trained from scratch on this relatively small dataset, did not provide beneficial representations for this task. It's possible that the dataset size was insufficient for effective Word2Vec training, or the specific hyper-parameters chosen for Word2Vec training were not optimal. In contrast, the randomly initialized embeddings, which were learned during the model training process, proved to be much more effective.

Model Evaluation

The models were evaluated using several key metrics:

- Accuracy: The overall proportion of correctly classified messages.
- Confusion Matrix: Provides a detailed breakdown of correct and incorrect predictions.

- True Positives (TP): Correctly predicted spam messages.
- True Negatives (TN): Correctly predicted ham messages.
- False Positives (FP): Ham messages incorrectly predicted as spam.
- False Negatives (FN): Spam messages incorrectly predicted as ham.
- Precision: The proportion of positive identifications that were actually correct ($TP / (TP + FP)$).
- Recall: The proportion of actual positives that were correctly identified ($TP / (TP + FN)$), also known as sensitivity.
- F1-Score: The harmonic mean of precision and recall, providing a single metric that balances both.

Evaluation Summary:

- Simple RNN: Achieved high accuracy (0.9758), with strong precision (0.98) and recall (1.00) for ham. For spam, it had a precision of 0.98 and a recall of 0.84, meaning it missed some spam messages (false negatives).
- LSTM: Outperformed Simple RNN, with an accuracy of 0.9821. It maintained excellent performance for ham (0.98 precision, 1.00 recall) and improved spam detection with a precision of 0.99 and a recall of 0.87.
- Word2Vec + LSTM: This model struggled significantly. While its accuracy was 0.8664 (due to correctly classifying most ham messages), its recall for spam was 0.00, indicating it did not identify a single spam message correctly. This is evident from the error analysis where it misclassified actual spam messages as ham.

Conclusion and Future Work

This project successfully explored the application of deep learning models for SMS spam/ham classification. The **LSTM model emerged as the best performer**, achieving the highest accuracy and a robust balance of precision and recall for both classes. It demonstrated a superior ability to identify spam messages compared to the Simple RNN. A key finding was the surprising underperformance of the Word2Vec + LSTM model, suggesting that training Word2Vec embeddings on this specific dataset was detrimental rather than beneficial.

Limitations: One limitation observed was the poor performance of the Word2Vec + LSTM model, likely due to the dataset size or sub-optimal Word2Vec configuration. There's also the inherent challenge of subtle spam messages that might bypass the current models.

Future Work: To further enhance the model's capabilities, several avenues can be explored:

- **Hyperparameter Optimization:** Conducting a more extensive search for optimal hyperparameters (e.g., learning rate, number of LSTM units, dropout rates) could yield better results.
- **Larger Pre-trained Embeddings:** Instead of training Word2Vec from scratch, utilizing pre-trained word embeddings like GloVe or FastText (trained on much larger corpora) could provide more robust and generalizable word representations.
- **Ensemble Methods:** Combining predictions from multiple models (e.g., RNN and LSTM) could potentially lead to more stable and accurate classifications.
- **Data Augmentation:** Exploring techniques to augment the training data, especially for the spam class, to help the model learn more diverse spam patterns.
- **Attention Mechanisms:** Integrating attention layers into the LSTM architecture could allow the model to focus on more relevant parts of the message when making predictions, potentially improving accuracy and interpretability.

